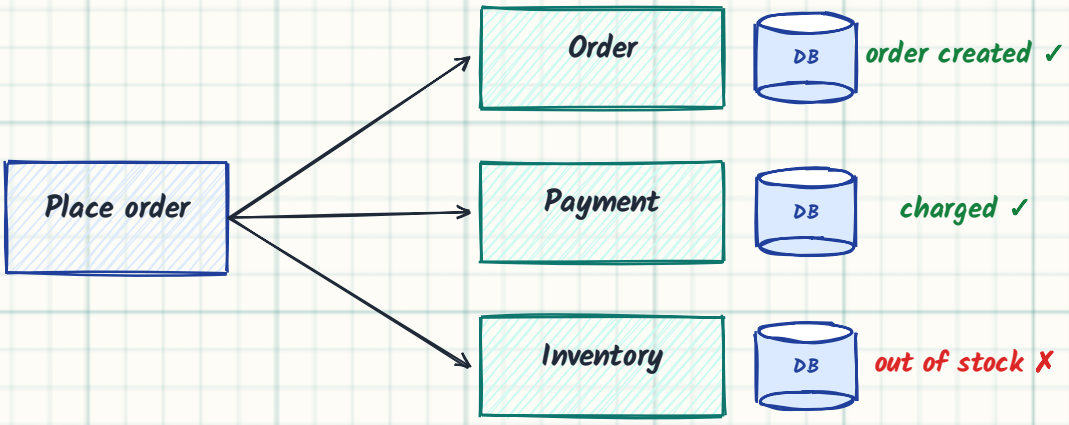
The Saga Pattern

Lesson 11 · distributed transactions without 2PC

In this lesson → ① The problem ② Why not 2PC ③ Saga pattern ④ Compensations ⑤ Choreography ⑥ Orchestration ⑦ States ⑧ Reliability ⑨ Interview Q&A

Running project: ShopNest, our e-commerce app (users, products, orders, payments, notifications).

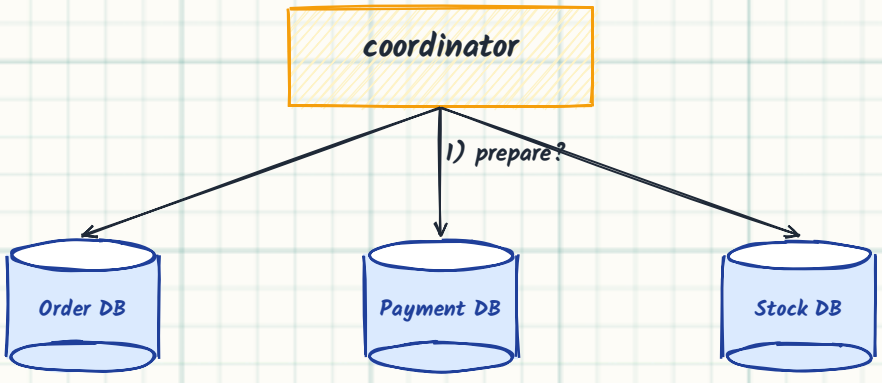
① The problem



money taken, no stock... now what?

- ✗ Each service has **its own database** (Lesson 10)
- ✗ No single ACID transaction across services
- ✗ Partial failure → **inconsistent** business state

② Why not two-phase commit (2PC)?



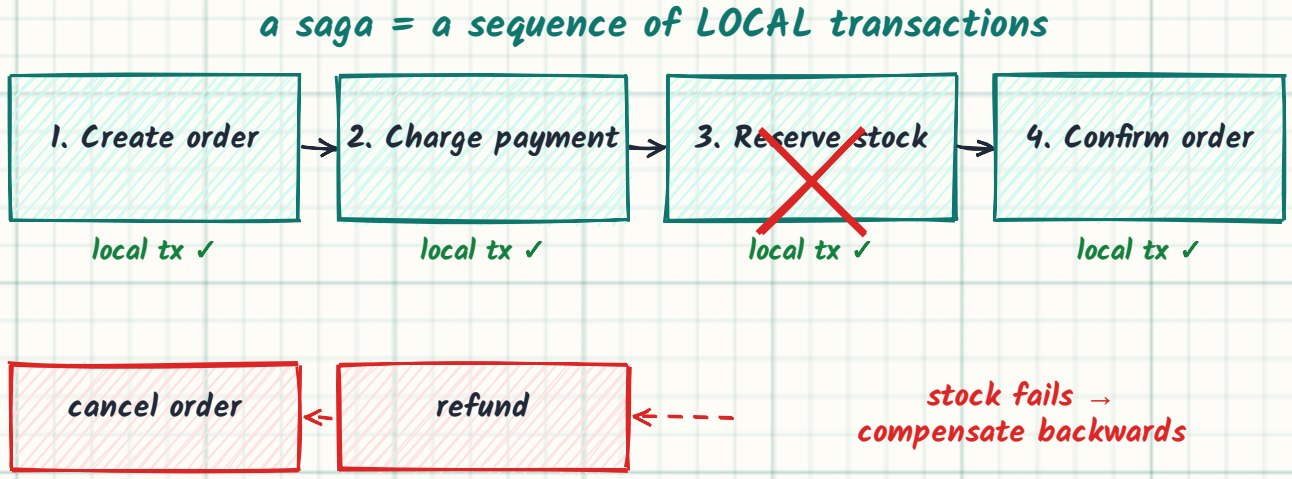
2) all "yes" → commit · any "no" → rollback

locks held while waiting · coordinator = single point of failure

✗ Locks held while waiting → poor throughput
✗ Coordinator = single point of failure
✗ Many NoSQL stores & brokers (e.g. Kafka as a participant) don't support XA
✗ Tight runtime coupling: all participants must be up

③ The Saga pattern

> **Saga** = a business transaction split into a sequence of local transactions. If a step fails, **compensating transactions** undo the completed steps in reverse order. (Garcia-Molina & Salem, 1987)

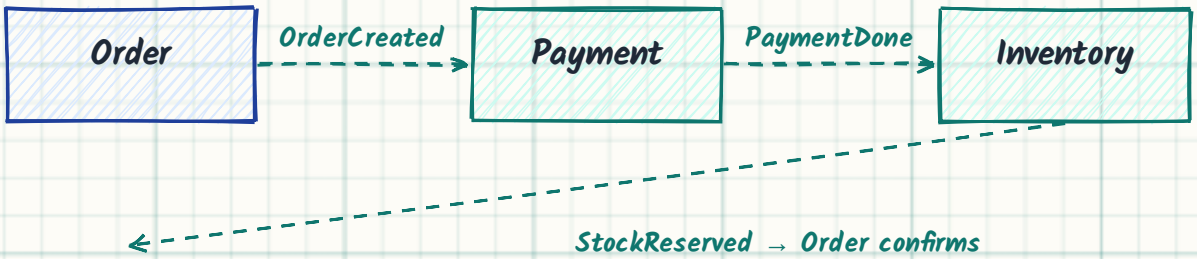


④ Compensating transactions

Step	Compensation
Create order (PENDING)	cancel order
Charge payment	refund
Reserve stock	release stock
Send email	cannot undo → do last / send apology

> **Remember!**
> A compensation is **not a rollback**. The step really happened; you run a **new** action that undoes its business effect.

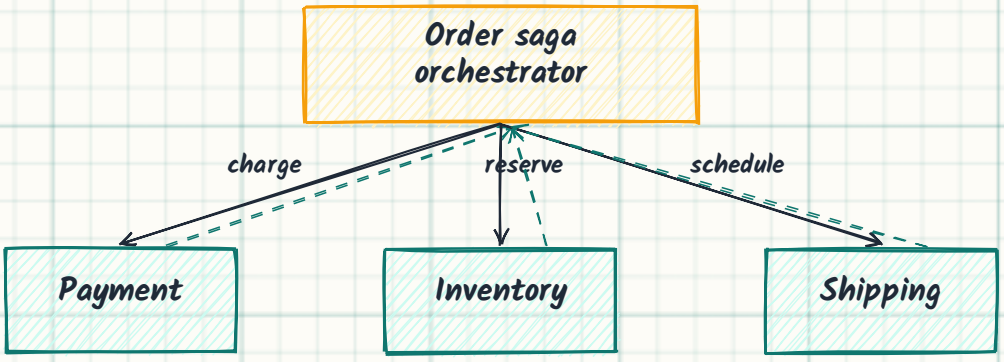
5 Choreography



no boss: each service reacts to events

- ✓ No central component
- ✓ Loose coupling
- ✓ Simple for 2–3 steps
- ✗ Flow spread across services
- ✗ Hard to see & debug
- ✗ Risk of cyclic event chains

6 Orchestration

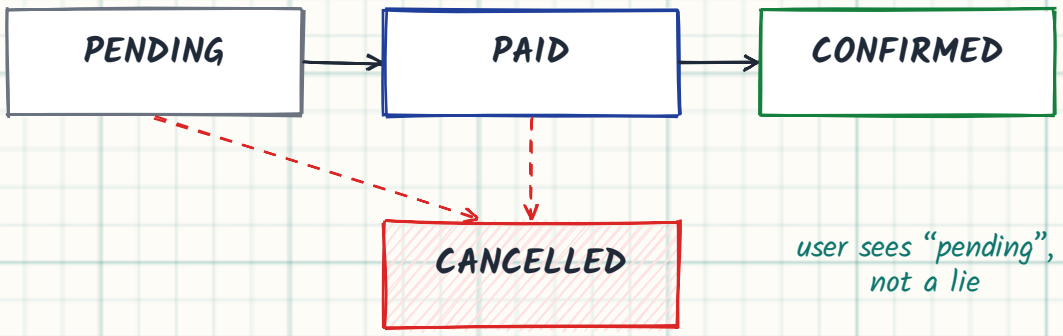


one place knows the whole flow

- ✓ Whole flow in one place
- ✓ Easier to test & monitor
- ✓ Good for long flows
- ✗ Orchestrator can grow into a "god service"
- ✗ One more component to run

	Choreography	Orchestration
Control	events, no boss	central orchestrator
Coupling	looser	flow centralized
Visibility	spread out	one place
Best for	2–3 simple steps	long flows, branches
Tools	Kafka, RabbitMQ	Temporal, Camunda, AWS Step Functions

7 Eventual consistency & states



Model in-between states honestly: PENDING → PAID → CONFIRMED *or* CANCELLED. *The UI shows "pending".*

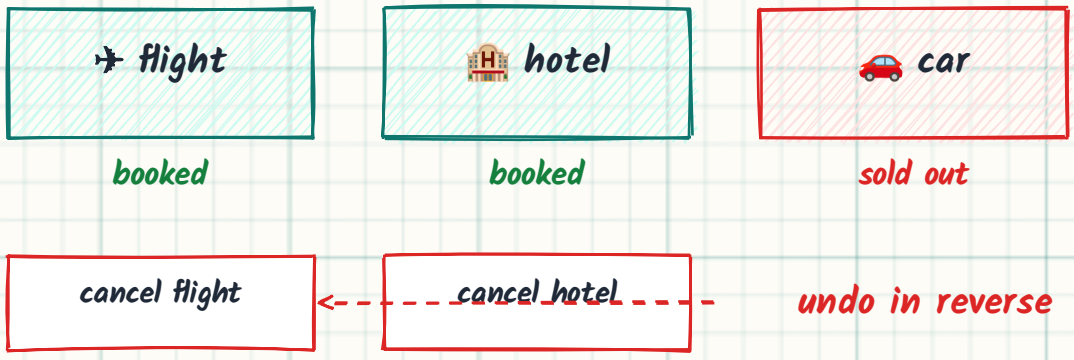
8 Making sagas reliable

1. Make steps & compensations **idempotent** (messages can repeat)
2. Publish events **atomically** with the DB change → transactional outbox (Lesson 13)
3. Handle **isolation**: pending states, semantic locks, re-read values

9 In code: a choreography step

```
@KafkaListener(topics = "inventory-events", groupId = "order")
void on(StockReservationFailed e) {
   Order o = orders.find(e.orderId());
   o.cancel("OUT_OF_STOCK");            // local tx
   publisher.publish(new OrderCancelled(o.id())); // Payment refunds
}
```

10 Analogy: booking a trip



Trip saga
Flight ✓, hotel ✓, car ✗ → cancel hotel, cancel flight. Same idea as order → payment → stock.

11 Design tips

- → Put **irreversible** steps (emails, shipping) **last**
- → Use a **saga ID** / correlation ID on every message
- → Store saga state so it can **resume** after a crash
- → Add timeouts: a step with no reply must fail or retry

⑫ Interview Corner

Q. How do you handle a transaction spanning several microservices?

A. Saga pattern: a sequence of local transactions with compensating transactions on failure.

Q. Why not use two-phase commit?

A. Holds locks, coordinator is a single point of failure, many stores/brokers don't support it, tight coupling.

Q. Choreography vs orchestration?

A. Choreography: services react to events, no central control. Orchestration: a coordinator sends commands and tracks state.

Q. What is a compensating transaction?

A. A new action that semantically undoes a completed step (refund, release stock).

Q. What consistency do sagas give?

A. Eventual consistency; model pending states and make steps idempotent.

⚡ Revise in 60 seconds

- → No cross-service ACID → **Saga**
- → Local tx per step
- → **Compensate** in reverse
- → Irreversible steps last
- → **Choreography**: events
- → **Orchestration**: coordinator
- → Pending states · idempotency
- → Outbox for reliable events

Next lesson → MS-12
CQRS & Event Sourcing.
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**

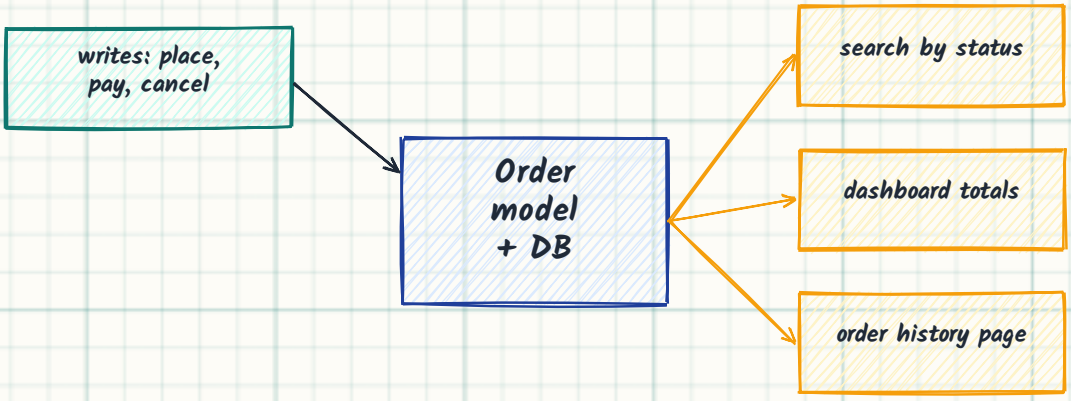
CQRS & Event Sourcing

Lesson 12 · separate reads from writes, store every change

In this lesson → ① One-model problem ② CQRS ③ Why & cost ④ Event sourcing ⑤ Rebuilding state ⑥ Projections ⑦ Pros & cons ⑧ When to use ⑨ Interview Q&A

Running project: ShopNest, our e-commerce app (users, products, orders, payments, notifications).

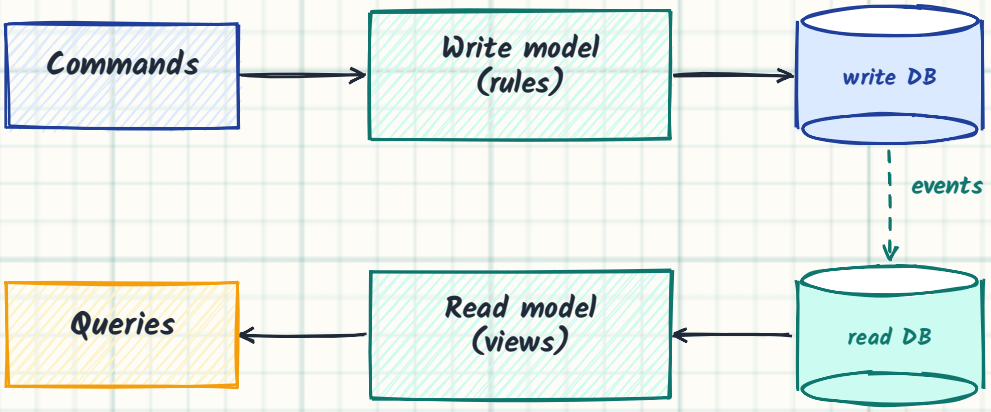
① The problem: one model for everything



one model tries to do everything → slow & messy

② CQRS

> **CQRS** (Command Query Responsibility Segregation, named by Greg Young) = separate models for writes (commands) and reads (queries), usually kept in sync by events.



	Write side	Read side
Handles	commands: PlaceOrder, Pay	queries: search, history, dashboard
Model	rules, aggregates (Lesson 3)	denormalized views
Storage	normalized SQL	view tables, search index, cache
Consistency	strong (ACID)	eventual

- ✓ Scale reads & writes separately
- ✓ Best storage per side
- ✓ Clean write model
- ✗ Eventual consistency on reads
- ✗ More moving parts
- ✗ Overkill for simple CRUD

③ Event sourcing

> **Event sourcing** = store every state change as an immutable event; the event log is the source of truth, and current state is rebuilt by replaying it.

CRUD: store the latest

balance = 700

how did we get here? 🤷

Event sourcing: store every change

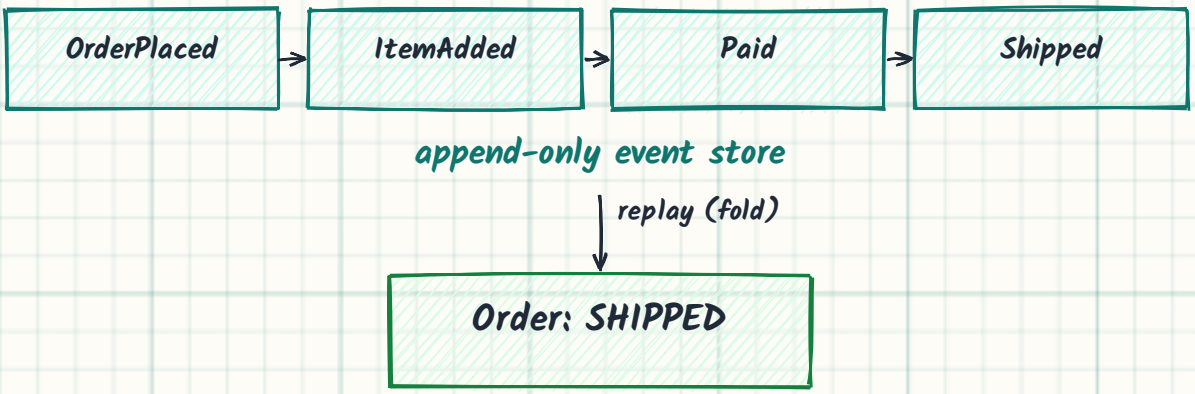
Deposited 1000

Withdrew 200

Withdrew 100

replay → balance = 700

④ Rebuilding state



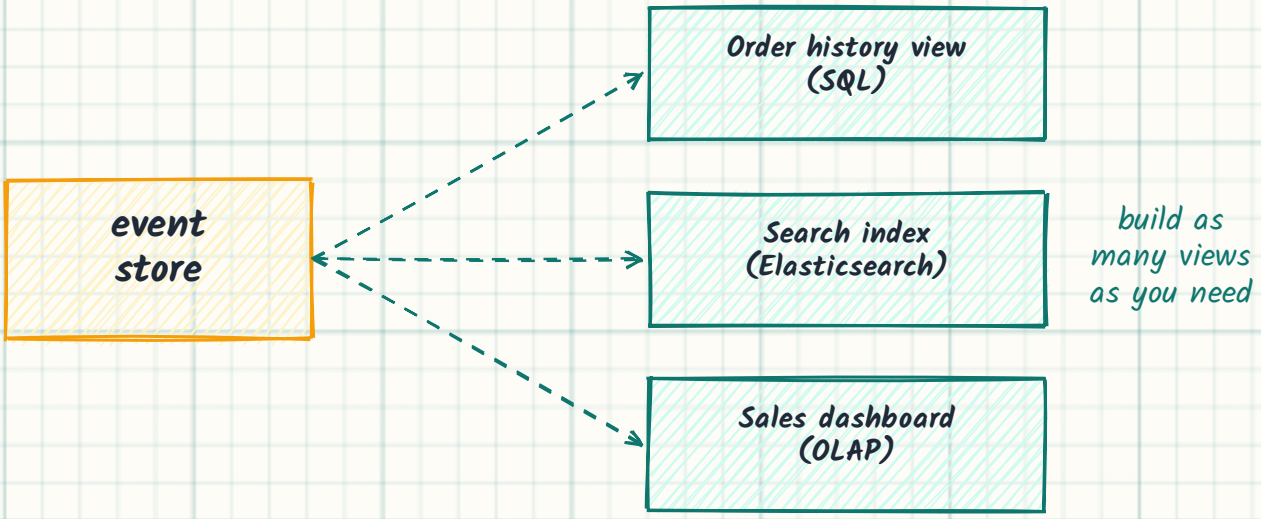
current state = replay of all events

→ Events are **append-only**: never update or delete
→ Fix mistakes with a **correcting event**
→ Long streams → take **snapshots** (e.g. every 100 events)

```
class Account {
  private long balance;
  // current state = fold of past events
  static Account from(List<Event> events) {
    Account a = new Account();
    events.forEach(a::apply);
    return a;
  }
  void apply(Event e) {
    switch (e) {
      case Deposited d -> balance += d.amount();
      case Withdrew w  -> balance -= w.amount();
    }
  }
}
```

5 Projections

> **Projection** = a read model built by processing the event stream; you can build many, and rebuild any of them by replaying.



6 Pros & cons of event sourcing

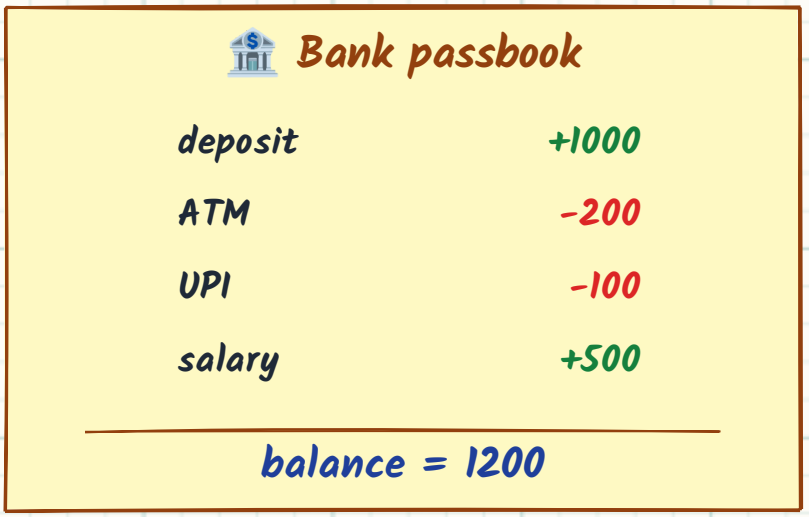
- ✓ Full audit log
- ✓ Time travel: state at any past moment
- ✓ Replay → new views, debugging
- ✓ Natural fit for event-driven systems
- ✗ Steeper learning curve
- ✗ Snapshots needed for long streams
- ✗ Event schema versioning
- ✗ Ad-hoc queries need projections

7 When to use them

Pattern	Use when	ShopNest example
CQRS	reads very different from writes, heavy read load	product catalog, sales dashboard
Event sourcing	history, audit, "what happened when" matter	payments ledger, order timeline
Neither	simple CRUD	user profile, addresses

Not the same thing
CQRS and event sourcing are **often combined**, but each can be used on its own.

8 Analogy: a bank passbook



⑨ Interview Corner

Q. What is CQRS?

A. Separating the write model (commands) from the read model (queries), often synchronized through events.

Q. What is event sourcing?

A. Persisting every change as an immutable event and rebuilding current state by replaying events.

Q. Are CQRS and event sourcing the same?

A. No. They complement each other but each can be used alone.

Q. What is a projection?

A. A read model derived from the event stream, rebuildable by replay.

Q. What are snapshots?

A. Saved state at a point in the stream so you don't replay everything from the start.

⚡ Revise in 60 seconds

- → **CQRS**: write model ≠ read model
- → Events sync read side
- → Reads eventually consistent
- → Use where reads ≠ writes
- → **Event sourcing**: store every change
- → Replay → current state
- → **Projections** = read views
- → Snapshots · versioning

Next lesson → MS-13
Transactional outbox, idempotency & exactly-once myths.
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**

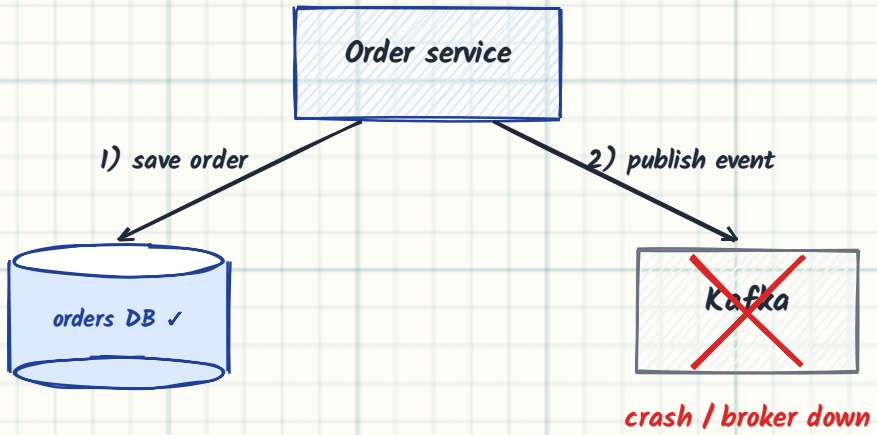
Outbox & Idempotency

Lesson 13 · no lost events, no double charges

In this lesson → ① Dual write problem ② Why simple fixes fail ③ Transactional outbox ④ Duplicates ⑤ Idempotent consumers ⑥ Idempotency keys ⑦ Exactly-once myth ⑧ Checklist ⑨ Interview Q&A

Running project: ShopNest, our e-commerce app (users, products, orders, payments, notifications).

① The dual write problem



order saved, event lost → others never know

> **Dual write** = updating the database and publishing to a broker as two separate operations. A crash between them leaves the system inconsistent.

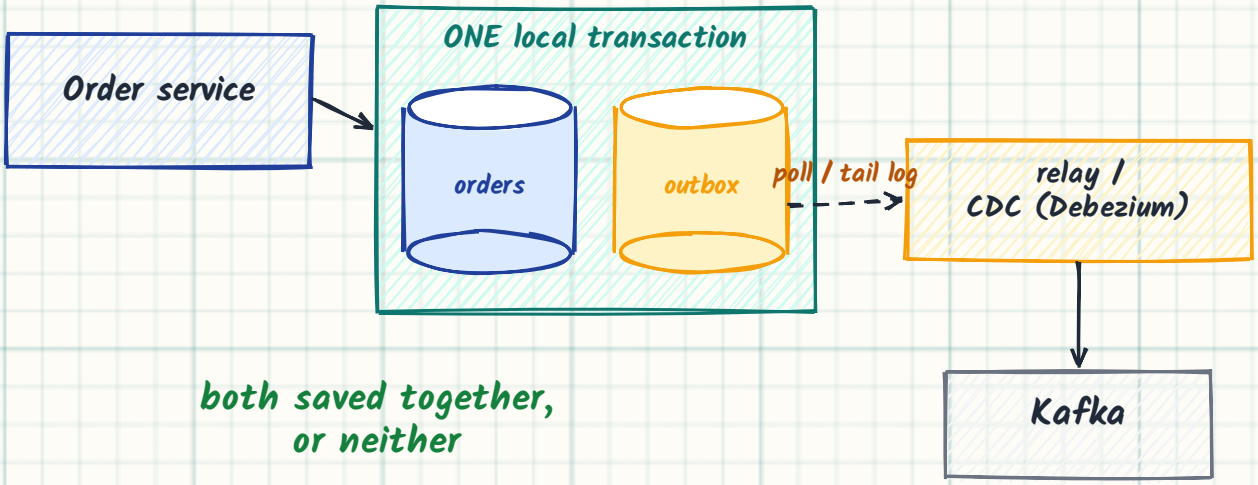
✗ Save then publish → event lost if the service crashes

✗ Publish then save → event for an order that doesn't exist

✗ 2PC across DB + broker → slow, fragile, often unsupported

② The transactional outbox

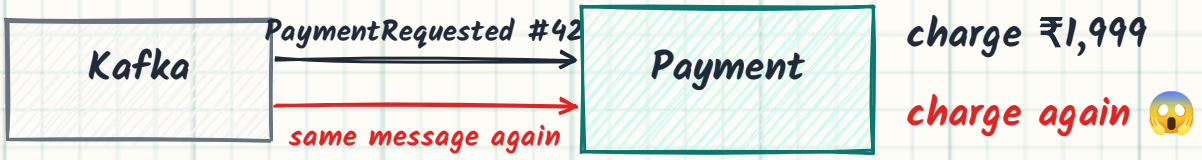
> **Transactional outbox** = write the business change **and** an event row into an outbox table in one local transaction; a relay later publishes outbox rows to the broker.




```
-- ONE local transaction
BEGIN;
INSERT INTO orders (id, status, total) VALUES (42, 'PLACED', 1999);
INSERT INTO outbox (id, aggregate_id, type, payload)
  VALUES (gen_random_uuid(), 42, 'OrderPlaced', '{"orderId":42}');
COMMIT;
-- a relay (poller or Debezium CDC) publishes outbox rows to Kafka
```

Relay style	How	Notes
Polling publisher	SELECT unsent rows every second	simple; adds DB load
Log tailing (CDC)	read DB transaction log (Debezium)	low latency; more infra

③ But now: duplicates

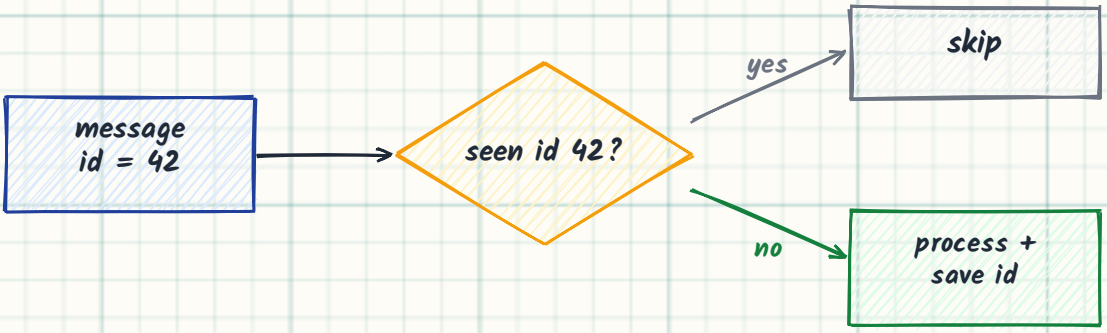


at-least-once delivery → duplicates WILL happen

→ Relay publishes, crashes before marking "sent" → publishes again
→ Consumer processes, crashes before committing offset → re-reads
→ Result: **at-least-once** delivery → duplicates WILL happen

④ Idempotent consumers

> ***Idempotent*** = doing it twice has the same effect as once.

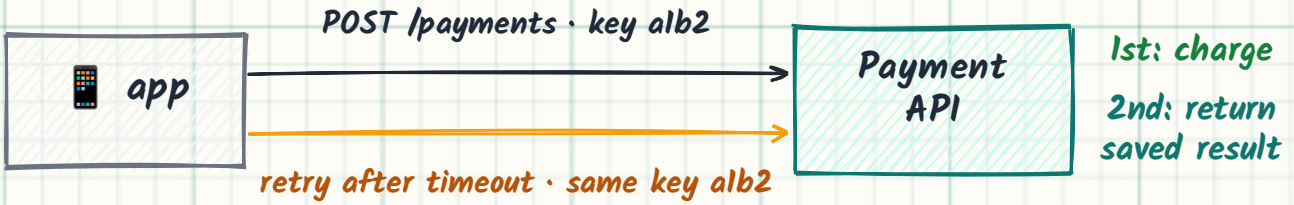


same id twice → same result once

```
@KafkaListener(topics = "payment-requests", groupId = "payment")
@Transactional
void on(PaymentRequested m) {
  if (!processed.insertIfAbsent(m.messageId())) return; // unique key
  payments.charge(m.orderId(), m.amount());
}
```

Technique	How
Processed-message table	store message ID with a unique key, same tx as the change
Natural idempotency	SET status = PAID (not balance += x)
Version / sequence check	ignore events older than the stored version
Idempotency key (APIs)	client sends a key; server stores & replays the result

⑤ Idempotency keys for APIs



Real world
Payment APIs such as Stripe accept an `Idempotency-Key` header so client retries never double-charge.

6 The exactly-once myth

Guarantee	Reality
Exactly-once across DB + broker + external API	not practical
Kafka exactly-once semantics	only for read-process-write inside Kafka
At-least-once + idempotency	**effectively once**: the standard recipe

7 Checklist

- ✓ No dual writes: use an **outbox** (or CDC)
- ✓ Every message has a unique **ID**
- ✓ Every consumer is **idempotent**
- ✓ Every retried API call carries an **idempotency key**
- ✓ Failing messages → retries → **DLQ** (Lesson 6)

Where it matters most
Payments, stock reservation, emails and anything that moves money or sends messages to users.

⑧ Interview Corner

Q. What is the dual write problem?

A. Writing to a database and a message broker in two separate operations; a failure between them causes inconsistency.

Q. How does the transactional outbox solve it?

A. The event is stored in an outbox table in the same local transaction as the business change, then published by a relay or CDC.

Q. What is an idempotent consumer?

A. One that gives the same result if a message is processed more than once, typically by tracking processed message IDs.

Q. What is an idempotency key?

A. A client-generated key sent with a request so the server can detect retries and return the original result.

Q. Can we have exactly-once delivery?

A. Not practically across systems. Use at-least-once delivery plus idempotent processing.

⚡ Revise in 60 seconds

- → **Dual write** = DB + broker separately
- → **Outbox**: same local tx
- → Relay: polling or **CDC** (Debezium)
- → Outbox → at-least-once
- → Duplicates **will** happen
- → **Idempotent** consumers (message IDs)
- → APIs: **idempotency keys**
- → At-least-once + idempotent = effectively once

Next lesson → MS-14
Fault tolerance: **circuit breaker, retry, timeout, bulkhead**.
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**

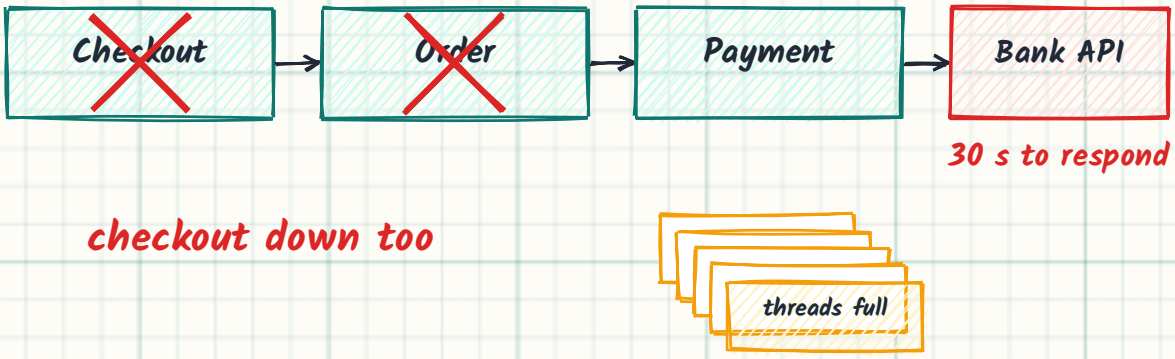
Fault Tolerance

Lesson 14 · timeouts, retries, circuit breakers & bulkheads

In this lesson → ① The problem ② Timeouts ③ Retries & backoff ④ Circuit breaker ⑤ Bulkheads ⑥ Fallbacks ⑦ Resilience4j ⑧ Putting it together ⑨ Interview Q&A

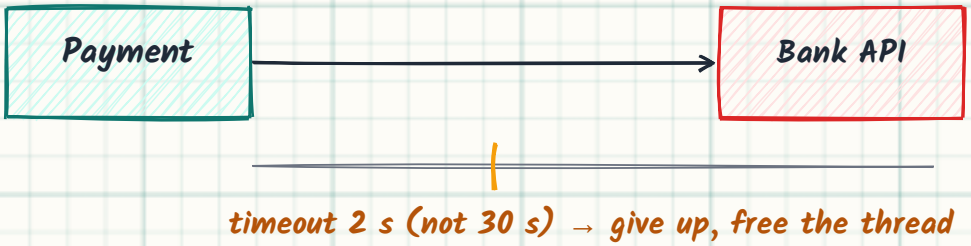
Running project: ShopNest, our e-commerce app (users, products, orders, payments, notifications).

① The problem: one slow dependency



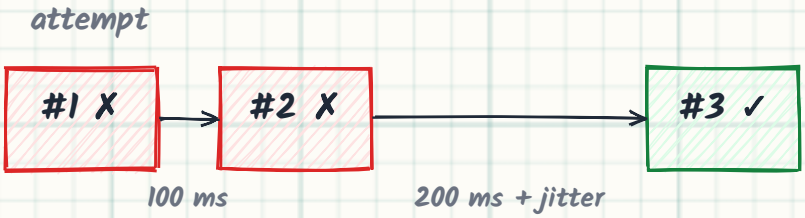
- ✗ Slow calls **hold threads**
- ✗ Callers wait on callers → **cascading failure** (Lesson 5)
- ✗ Retrying blindly makes it **worse**

② Timeouts



- ✓ Set **connect** and **read** timeouts on every call
- ✓ Inner timeouts shorter than outer ones
- ✓ A fast failure beats a slow one

③ Retries with exponential backoff

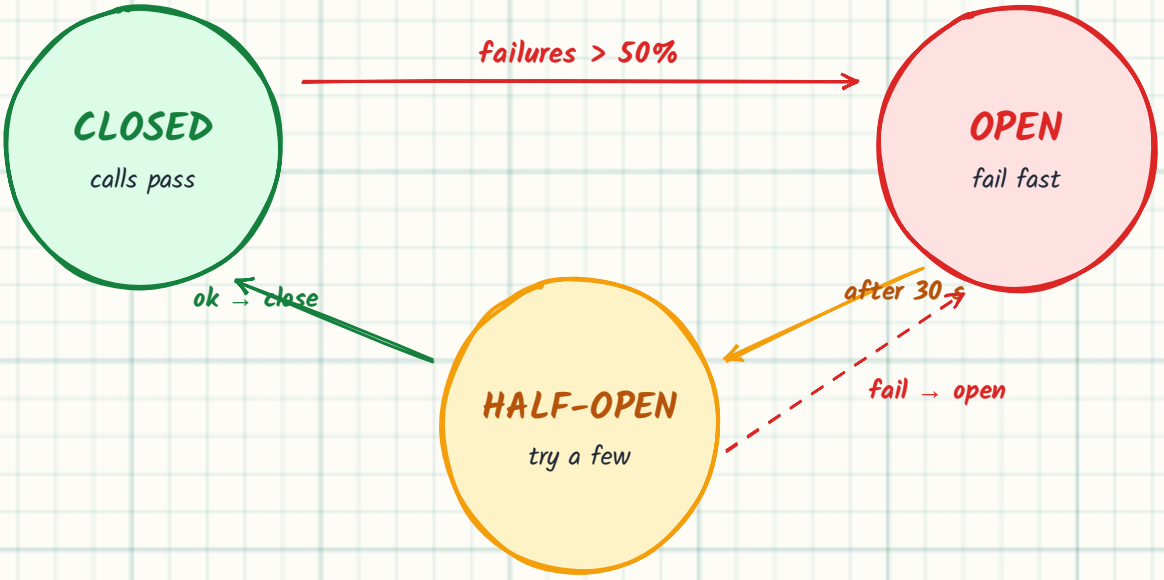


exponential backoff: wait longer each time
only retry idempotent calls · cap the attempts

Rule	Why
Only retry idempotent calls	avoid double charges (Lesson 13)
Exponential backoff	give the service time to recover
Add jitter	avoid thousands of clients retrying together
Cap attempts (e.g. 3)	prevent retry storms
Retry only transient errors	503, timeouts, not 400 bad request

④ Circuit breaker

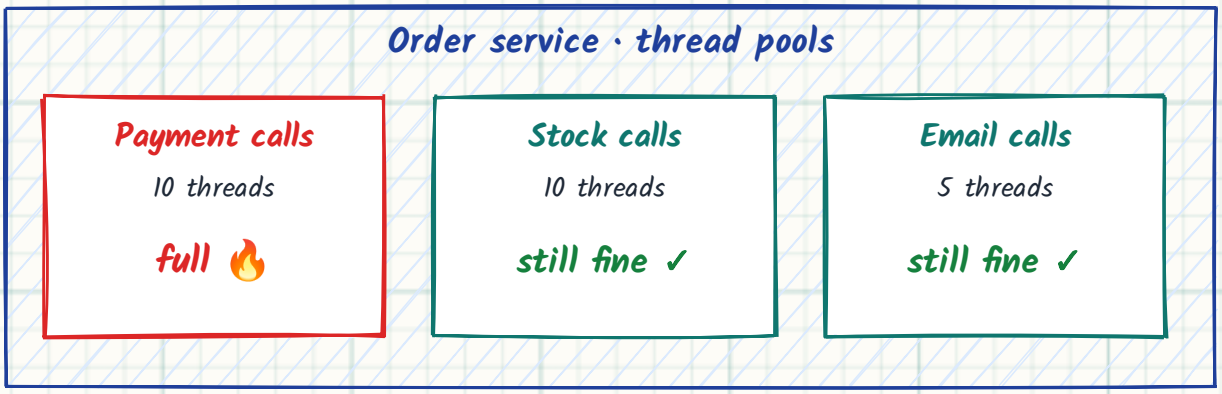
> **Circuit breaker** = wraps calls to a dependency; after too many failures it opens and fails fast, then tests recovery before closing again. (Popularized by Michael Nygard, *Release It!*)



State	Behaviour	Moves when
CLOSED	calls pass, failures counted	failure rate > threshold → OPEN
OPEN	calls fail immediately	wait duration passed → HALF-OPEN
HALF-OPEN	a few trial calls	success → CLOSED · failure → OPEN

Analogy
A **house fuse**: a fault trips it so the house doesn't burn; you reset it when the problem is fixed.

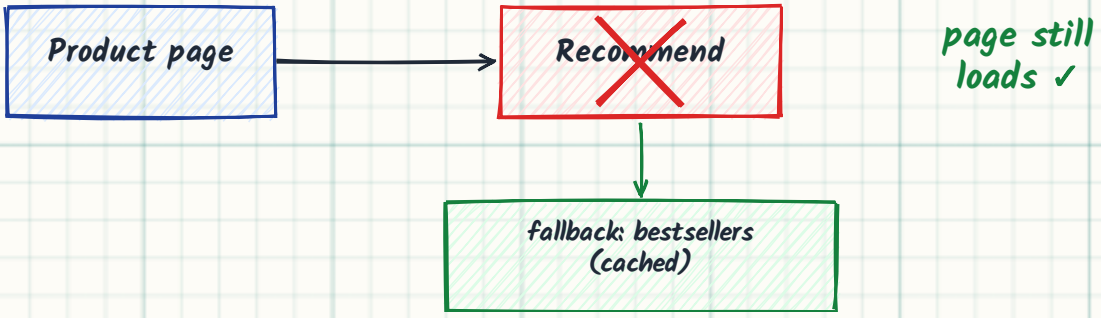
5 Bulkheads



like watertight compartments in a ship

- → Separate **thread pools** or **semaphores** per dependency
- → Also: separate **connection pools**, separate instances for critical traffic
- → One slow dependency can only exhaust **its own** pool

6 Fallbacks



Fallback	Example
Cached data	last known bestsellers
Default value	empty recommendations list
Queue for later	accept order as PENDING, pay later
Friendly error	"try again in a minute"

7 Resilience4j in Spring Boot

```
@CircuitBreaker(name = "bank", fallbackMethod = "later")
@Retry(name = "bank")
@TimeLimiter(name = "bank")
CompletableFuture<Receipt> charge(Payment p) {
  return CompletableFuture.supplyAsync(() -> bank.charge(p));
}
CompletableFuture<Receipt> later(Payment p, Throwable t) {
  return completedFuture(Receipt.pending(p.id()));  // retry async
}

# application.yml
resilience4j.circuitbreaker.instances.bank:
  failure-rate-threshold: 50
  wait-duration-in-open-state: 30s
resilience4j.retry.instances.bank:
  max-attempts: 3
  wait-duration: 200ms
  enable-exponential-backoff: true
```

→ Resilience4j modules: CircuitBreaker, Retry, TimeLimiter, Bulkhead, RateLimiter

→ Netflix **Hystrix** is in maintenance mode, so use Resilience4j

→ Service meshes (Istio) can also add timeouts, retries and outlier detection

8 Putting it together

1. **Timeout** limits each attempt
2. **Retry** handles short, transient failures
3. **Circuit breaker** stops calls when failures persist
4. **Bulkhead** limits concurrent calls
5. **Fallback** decides what the user sees

Interview Corner

Q. What is a circuit breaker?

A. A wrapper that stops calling a failing dependency after a failure threshold, fails fast while open, and tests recovery in half-open.

Q. Why exponential backoff with jitter?

A. To give the dependency time to recover and avoid synchronized retry storms.

Q. What is the bulkhead pattern?

A. Isolating resources (thread/connection pools) per dependency so one can't exhaust all.

Q. Timeout vs circuit breaker?

A. A timeout bounds one call; a circuit breaker decides whether to call at all based on recent failures.

Q. When should you NOT retry?

A. Non-idempotent operations without an idempotency key, and non-transient errors like 400 or 401.

⚡ Revise in 60 seconds

- → **Timeout** on every call
- → **Retry**: backoff + jitter + cap
- → Only retry **idempotent** calls
- → **Fallback**: degrade gracefully
- → **Breaker**: closed → open → half-open
- → Fail fast while open
- → **Bulkhead**: pool per dependency
- → **Resilience4j** in Spring

Next lesson → MS-15
Rate limiting, caching & scaling services.
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**